4.2.3 Implementing UI state management patterns

We will now refactor our existing Astra AI project to use these powerful new tools. The key architectural upgrades are replacing `streamText` with `streamUI` and introducing `createAI`. These changes will transform our basic chat application into a more robust and flexible system capable of handling complex, dynamic responses and managing state more effectively.

> **NOTE** The source code for the examples in this chapter is available at https://github.com/Generative-AI-Web-Apps/Code/tree/main/ch04/chat-rsc-stream-ui.

Project code: chat-rsc-stream-ui

The code for this example project is in the ch04/chat-rsc-stream-ui folder. You can use that example by running this command in the source folder:

```
$ npm run dev -w ch04/chat-rsc-stream-ui
```

You'll also need a Google API key to access their REST API. The appendix offers instructions on how to set up and create a secret key for Google AI and other providers:

```
// .env
GOOGLE_API_KEY=<SECRET_KEY>
```

For details on setting up and creating API keys, refer to the appendix.

To address these needs, we can use two key architectural upgrades that will improve our state management and response-handling capabilities:

- *Replacing* `streamText` *with* `streamUI`—This helper allows us to stream responses not just as text, but as fully fledged React components.
- *Introducing* `createAI`—This is a comprehensive solution that unifies UI and AI states, providing a single source of truth for our entire application.

These upgrades are focused on improving the underlying architecture of our state management system, and they do not affect the visual representation of the application. By implementing these three key upgrades, we'll transform our basic AI chat application into a more robust, flexible system. Figure 4.9 visually showcases the application architecture before and after the upgrades and changes from `streamText` to `streamUI` and the introduction of `createAI`.

Before the upgrade, the Astra AI project used the `streamText` helper function for managing AI responses. This function streamed text-based responses from the AI model, which were then handled by the React components on the client side. State management was primarily done using React's `useState`.